

Tue 4-Feb-2025

3:04pm

Issues I'm thinking about and possible solutions: A few were mentioned/alluded to in class:

- What happens if a user is deleted and they have outstanding messages? - I think we should simply delete all messages with the user (i.e., cascading deletions in database terms)
- What happens if (1) a message is deleted and has not been shown to the recipient yet, or (2) has already been delivered to the recipient? - Both cases are a little messy. I think it's unclear in (1) how we intervene to stop delivery of the deleted message. I think in both cases a simple solution may be to, upon receiving a deletion request from one client, pass through the socket to the other client a request for the client to stop showing the deleted message to the user. This means deleted messages aren't really deleted in some sense, but this behavior seems pretty reasonable.

Things Savanna (my programming partner) and I have agreed upon:

- Programming everything in Python
- MUST SECURELY HASH PASSWORDS ON THE SERVER (see below for client side hashing)

Notes to discuss with Savanna:

- I think we may want to have different
- DO NOT TRANSMIT UNHASHED PASSWORDS OVER THE NETWORK (else we commit the "network is secure" fallacy). Always hash passwords client side before transmitting

5:05pm

Meeting with Savanna:

We met to sketch out how communication via the sockets would go.

We agreed that the server should manage a source of truth for information in the distributed system, and also manage instantly passing around information throughout the system.

Design sketches:

- Create account
- Log in to account
- List accounts
- Send message
- Receive msg
- Delete msg
- Delete acct

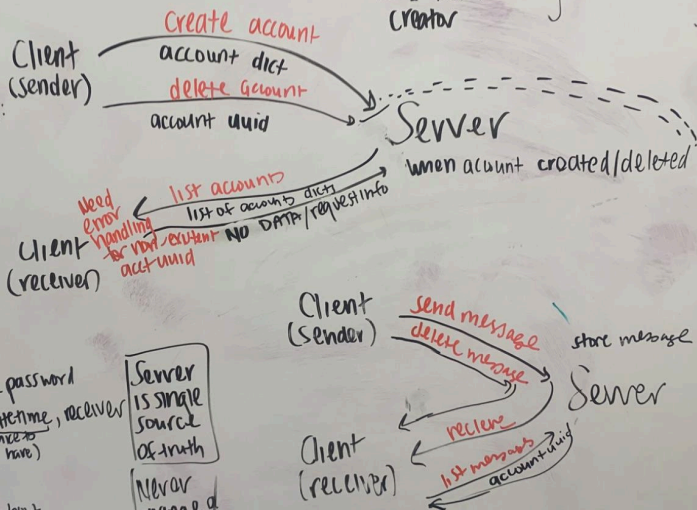
Possible issues:

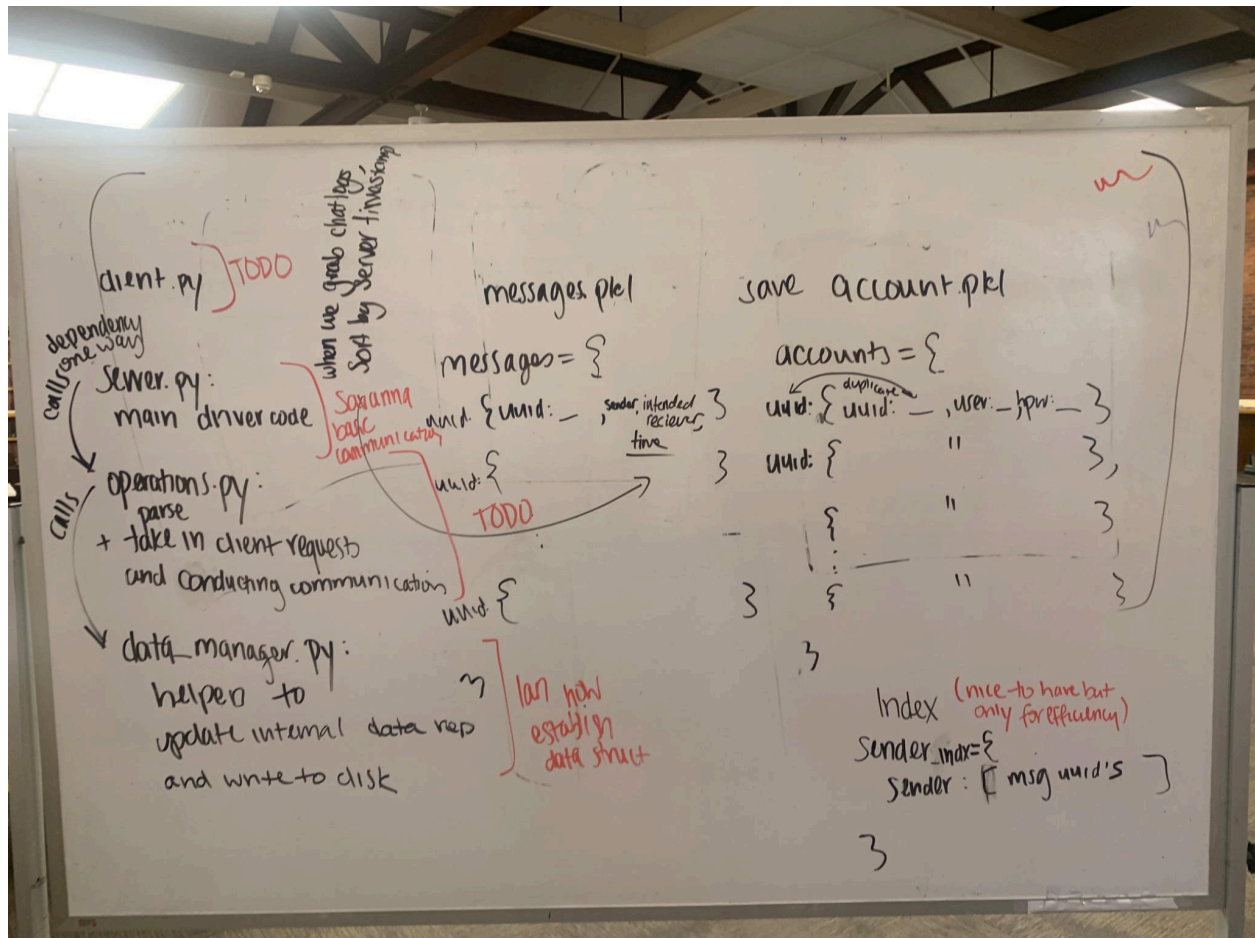
Account: uid, username, hashed_password
 Message: uid, sender, content, datetime, receiver
 (account uid) (link to here)

store pickle file
 (persistent storage)

Since a pickle
 Server loads pickle on boot
 Every op write to pickle

Server is single source of truth
 Never change a value





10:15pm

Implementation of account data management functions:

I implemented functions in **accounts.py** that let us manage our account data.

- There are functions to create accounts, delete accounts, and list accounts.
- Two helpers manage persisting accounts on the disk: one is able to load accounts from a pickle file, and one is able to write back to the pickle file. Whenever we make updates we make sure the pickle file is updated.

Thu 6-Feb-25
9:26pm

Discussion with Savanna on whether to do a SQL database for persistent data store:

Upsides:

- A simple `sqllite` database comes with a lot of tools that would make data storage pretty easy
- SQL databases naturally have efficiency like indexes for queries built in, so it would help streamline our code

Downsides:

- It does complexify the project
- Requires refactoring some code, but we designed our code bas so the data management and persistence is abstracted away from the application logic, so this refactor should be very easy

We ultimately decided to refactor using `sqllite`

Our plan to make further progress:

- Savanna will work on implementing log in, passwords, authentication, etc.
- I will work on implementing our data store with SQL, additional capabilities like searching for wildcard users, pagination of users, searching messages by sender/receiver

I have also implemented the beginnings of `messages.py` which handles the storage processes for messages similarly to how `accounts.py` manages the storage processes for accounts.

Fri 7-Feb-25

3:21pm

I have refactored the accounts data management functions to use a sqlite database and implemented unittests. Though I will need further unittests to ensure robust coverage, but this is a good start.

I am working on doing the same for messages.

I'm currently working on the `sql` branch while Savanna works on authentication on the `main` branch, so that way we don't have any nasty conflicts and we can merge our code when we meet later.

Another note re: password hashing - I thought about it and I think I was wrong about the importance of hashing passwords client side. If the hashed password is intercepted in transit over the network, it could still be supplied to the server and used to forge requests. So really we are reliant on the security of the network to transmit passwords whether we choose to hash client or server side (or both, but that would be tacky). This is a limitation we will need to mention in our documentation.

Sat 8-Feb-25

7:05pm

So far Savanna and I have been working with our wire protocol. We have now discussed implementing a `JSON_MODE` so that communication is done in JSON.

Though I was able to implement SQL for the accounts and it would have been easy enough to do so for the messages, we have realized that it does complexify the system a bit on the server side, and the text files are actually working very robustly.

We ultimately talked it over and decided to keep the system simple by keeping our text file approach. Savanna was also able to implement a methodology to cache pending messages in a text file. Our data storage is fully persistent.

I completed implementing JSON mode on the server side, but still need to do so for the client.

Sun 9-Feb-25
7:07pm

I was waiting for Savanna to complete some work on the GUI which required changing how the client code manages communication. (The design is that `client.py` handles communication with the server, and `gui.py` handles display. This abstracts away GUI code from communication code, so we could change the nature of the communication/wire protocol without much issue because of this abstraction.)

Since she completed that work, I have been able to implement JSON mode on the client side as well.

I am beginning implementing strong tests.

Our goal is to have all the areas of the program robustly tested. I have done tests for the accounts and messages logic.

We are now turning to the server, client, and GUI. After some discussion, Savanna and I agreed that we need robust tests for all functionality, including communication and user interface elements.

Tinker interfaces well enough with unittests, but testing the client and server in unittests will require spinning up a test server or client respectively or simulating a socket to ensure correct communication. I have a plan for doing so by using simulation set up and tear down code in the unittests and will begin work on this.

Mon 10-Feb-25

11:11pm

I have managed to get tests written for the client by simulating a server. We are able to robustly test all client functionality and communication. Surprisingly, our code has been working elegantly and the test engineering process uncovered few bugs save minor issues.

I have begun working on the server tests, but this has been a bit trickier. Simulating a client for communication is a little harder, especially because clients need to pass around some data to the server.

I will continue working on simulating these requests.

Tue 11-Feb-25

10:05pm

I have finally managed to get the server tests working. There is now a complete test suite simulating communication with a client and checking that all communication is correct.

There were some issues with variables being out of scope in the server tests, because they were declared in main in the `server.py` file which wasn't working, but I have moved the variable declarations and gotten everything functional.

I also implemented comprehensive GUI tests to test for the functionality of UI elements.

With this, we have total test coverage.

I then turned to writing documentation. I have been able to document installation, some key designs of the program and the organization of the code, and other critical points in our [readme.md](#) so that users will understand how to use it and there is good documentation of how the code is laid out so various features can be easily found.

Savanna also worked to organize the functions nicely. I have documented the unittests thoroughly so it is clear what is intended to be tested in each unittest.

We also tested our wire protocol vs. the JSON implementation. Savanna was able to adapt my client/server tests to have the code to measure this. See Savanna's notebook for more commentary on this.

Results of comparison:

```
Test Summary
-----
4 tests run in total.
All tests passed!

Detailed Byte Size Totals (Application Payload Only):

Custom (Plain) Mode:
  Sent:      22 bytes
  Received:  22 bytes
  Total:     44 bytes

JSON Mode:
  Sent:      41 bytes
  Received:  49 bytes
  Total:     90 bytes
```